RAG EVAL – Document Ingestion Pipeline (V1 vs V2)

From text-only ingestion to multimodal ingestion with visual understanding and richer metadata

Version 1.0 – Text-Only Ingestion Pipeline

Supports: TXT, PDF, DOCX (text and tables only)

1 Document Upload / Discovery
Load documents from data/metrostate_documents or upload through browser.

TXT · PDF · DOCX

2 Text Extraction
Extract textual content using format-specific loaders.

- **TXT Loader** — Read text content
- **PDF Loader (PyMuPDF)** — Extract text and tables
- **DOCX Loader (python-docx)** — Extract text and tables

3 Chunking
Split extracted text into fixed-size chunks.

Chunk 1 (800 chars) → Chunk 2 (800 chars) → … → Chunk N (800 chars)

4 Create Embeddings
Generate text embeddings using all-MiniLM-L6-v2.

Sentence Transformer (all-MiniLM-L6-v2) → [0.12, -0.03, 0.45, ...]

5 Store in Databases
Save document, chunk, and embedding data.

- **MySQL**
  - Document records
  - Chunk text and metadata
  - Evaluation data
- **ChromaDB**
  - Vector embeddings
  - Similarity search index

Version 2.0 – Multimodal Ingestion Pipeline (Current)

Supports: TXT, PDF, DOCX, PNG, JPEG (text, tables, images, charts, diagrams)

1 Document Upload / Discovery
Load documents from data/metrostate_documents or upload through browser.

TXT · PDF · DOCX · PNG · JPG · JPEG

2 Format-Specific Processing and Element Extraction
Extract all document elements (text, tables, images, charts, diagrams) using the appropriate method.

- **TXT Processing**
  - Read text content
  - No visual elements
- **PDF Processing (PyMuPDF)**
  - Extract text
  - Extract tables
  - Extract images, charts, and diagrams
  - Use VLM for visual analysis
- **DOCX Processing (python-docx)**
  - Extract text
  - Extract tables
  - (Future: embedded images/diagrams)
- **PNG/JPEG Processing (Standalone Images)**
  - Validate image file
  - Use VLM for visual analysis
  - Generate detailed description
  - No text extraction needed

3 Element Types and Visual Analysis
Create structured elements with enriched metadata.

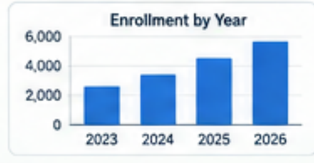
- **Text** — Paragraphs, headings, lists, etc.
  > Metro State University offers a variety of undergraduate programs...
- **Table** — Extracted table content

Term	Start Date
Fall 2026	Mar 23, 2026
Spring 2026	Oct 15, 2025

- **Image / Diagram** — VLM-generated description (using Gemini or Ollama)



- **Chart** — VLM-generated description



4 Chunking with Metadata
Create content-aware chunks and store rich metadata for each element.

Chunk 1 (Text) · Chunk 2 (Table) · Chunk 3 (Image description) · Chunk 4 (Diagram description) · …

Metadata stored with each chunk:
- Document ID and filename
- Source type (TXT, PDF, DOCX, PNG, JPEG)
- Content type (text, table, image, diagram)
- Page number (if applicable)
- Extraction method (e.g., pymupdf_table, ollama_vision)
- VLM model used (e.g., qwen2.5-vl:7b)

5 Create Embeddings and Store in Databases
Generate embeddings for all chunks and save with full metadata.

Sentence Transformer (all-MiniLM-L6-v2) → [0.12, -0.03, 0.45, ...]

- **MySQL**
  - Document records
  - Chunk text and metadata
  - Element types and provenance
  - Evaluation data
- **ChromaDB**
  - Vector embeddings
  - Similarity search index

Key Differences (V1 vs V2)

- V2 adds support for PNG and JPEG image ingestion.
- V2 uses a Vision-Language Model (Gemini or local Ollama) to analyze images, charts, and diagrams.
- V2 stores richer element metadata (source type, content type, page number, extraction method, model).
- V2 uses content-aware element extraction (text, tables, images, charts, diagrams) instead of text-only.
- V2 integrates image ingestion with the same chunking, embedding, and storage pipeline as text documents.
- V2 updates the frontend to allow PNG/JPEG uploads and displays multimodal indexing information.
- V2 includes new automated tests for image processing, ingestion, and frontend functionality.

What Remains (Planned for Next)

- Extend DOCX processing to extract and analyze embedded images, charts, and diagrams.
- Further improve visual element extraction and labeling.
- Evaluate multimodal retrieval and generation accuracy with additional test cases.
- Improve handling of temporary model API errors and fallback behavior.